

SOFTWARE CONSTRUCTION AND DEVELOPMENT

Engr. Muhammad Umer Haroon

SOFTWARE CONSTRUCTION AND DEVELOPMENT

WEEK 3

RECALL LAST 2 WEEKS

- SCD , Naming
- Variable declaration
- Decision statements
- print
- Formatted string , lists ,list operations , tuples

SET

- Set items are unchangeable, but you can remove items and add new items. No duplicate items. curly braces.

```
print("Learning Sets in Python")
example_set = {"cd70",125,'GS150','hayabusa'}
# every time it will display in random order
print(example_set)
#lets try duplicates
example_set = {"cd70",125,'GS150','hayabusa',125,125,'hayabusa'}
print(example_set)
```

Write its output

```
example_set2={True,1,1,0,0,True,False}
 print(example_set2)
```

Output : {0, True}

Explore and understand in lab

Functions & Reusability

Functions & Reusability

Reusability

- Reusability means designing the system so that you can reuse pieces of it in other systems.
- Reusability The extent to which and the ease with which you can use parts of a system in other systems.

Duplicated code is a **liability**. Every copy is a place bugs can hide and a place you must update.

No function code

```
# Version 1 No functions
students = ["Ali", "Sara", "Bilal"]
for name in students:
    print("=" * 30)
    print("Student:", name)
    print("=" * 30)

teachers = ["Dr. Khan", "Ms. Ayesha"]
for name in teachers:
    print("=" * 30)
    print("Teacher:", name)
    print("=" * 30)
```

Same code with function

```
def print_banner(label, name):
    print("=" * 30)
    print(f"{label}: {name}")
    print("=" * 30)

for name in ["Ali", "Sara", "Bilal"]:
    print_banner("Student", name)

for name in ["Dr. Khan", "Ms. Ayesha"]:
    print_banner("Teacher", name)
```

Three Levels of Reuse

Level	Description	Example
Within a file	Calling a function multiple times in one script	print_banner() above
Across files / modules	Importing a function into other programs	from utils import print_banner
Across projects / teams	Publishing a library or package	requests, numpy, pand as

Function vs routine vs method vs procedure

The terms function, routine, method, and procedure are often used interchangeably in programming, but they each have distinct meanings depending on the context and programming paradigm.

Function (generally)

- A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

FUNCTION

- A function is a routine that returns a value; a procedure is a routine that does not.

Routine

- The term routine is a more general term that can refer to any named block of code that can be called to perform a task. It encompasses both functions and procedures. In many contexts, "routine" is used synonymously with "subroutine," which is a broader term that includes any callable code segment, whether it returns a value or not

Those words routine and sub routines have mostly disappeared in modern languages. They were used in older programming languages. E.g Basic

Procedure

- In short, use a function if the primary purpose of the routine is to return the value indicated by the function name. Otherwise, use a procedure.

Method

- A method is similar to a function but is specifically associated with an object in object-oriented programming (OOP).
- It is defined within a class and usually operates on data contained within that class (the instance or class itself).
- The first parameter of a method in many languages (like Python) is typically self, which refers to the instance of the class.

When to Use a Function and When to Use a Procedure.

- Purists argue that a function should return only one value, just as a mathematical function does. This means that a function would take only input parameters and return its only value through the function itself.
- The function would always be named for the value it returned, as sin(), CustomerID(), and ScreenHeight() are.

Function example

```
def square(number):
    return number * number

# Using the function
result = square(5)
print("The square of 5 is:", result)  # Output: The square of 5 is: 25
```

Procedure example

```
def add_to_list(item, my_list):
    my_list.append(item)

# Using the procedure
my_numbers = [1, 2, 3]
add_to_list(4, my_numbers)
print("Updated list:", my_numbers)  # Output: Updated list: [1, 2, 3, 4]
```

	Function	Procedure
Returns a value?	Yes ,always returns something	No , performs an action, returns nothing (or void)
Purpose	Compute a result	Perform an action / side effect
Caller uses it by	Using its return value	Just calling it

This distinction is **strict in some languages** and **blurred in others**

- Pascal, Ada, Visual Basic — make a strict, formal distinction: function returns a value, procedure does not.
- C, C++, Java — everything is technically a "function," but one returning void behaves like a procedure.
- Python — every def is called a "function," even if it returns None (i.e., behaves like a procedure).

- So in Python, the words collapse into one but the design distinction still matters.
- You still decide: "Should this return a value, or should it just do something?"

Why to create them (function, procedure, method) ?

As an undergraduate computer-science student, I thought that the main reason to create a routine was to avoid duplicate code.

The introductory textbook I used said that routines were good because the avoidance of duplication made a program easier to develop, debug, document, and maintain.

Aside from syntactic details about how to use parameters and local variables, that was the extent of the textbook's coverage. It was not a good or complete explanation of the theory and practice of routines.

Valid Reasons to Create a Routine

1.Reduce complexity

- The main reason to create a routine is to simplify a program by hiding complexity. Once a routine is written, you can use it without needing to remember its internal details. This abstraction helps reduce code size, enhances maintainability, and improves correctness. Without routines, managing complex programs would be very challenging.

Code example

```
if value > 0:
    result = value * 2
else:
    result = 0

#instead do this
result = double_if_positive(value)
```

Avoid duplicate code

- The primary reason for creating routines is to avoid code duplication. When similar code appears in multiple routines, it indicates a flaw in code organization. To resolve this, you can extract the common code into a base class or a separate routine, allowing specialized routines to call this shared code. This approach reduces code size, simplifies modifications (since changes only need to be made in one place), and enhances reliability by minimizing the risk of inconsistent updates.

Code example

```
def print_rectangle_area(length, width):
area = length * width
print(f"The area of the rectangle is: {area}")

def print_square_area(side_length):
area = side_length * side_length
    print(f"The area of the square is: {area}")

print_rectangle_area(5, 3)
print_square_area(4)
```

```
def calculate_area(length, width):
return length * width


def print_rectangle_area(length, width):
area = calculate_area(length, width)  # Reusing the common function
    print(f"The area of the rectangle is: {area}")


def print_square_area(side_length):
area = calculate_area(side_length, side_length)  # Reusing the common function
    print(f"The area of the square is: {area}")
print_rectangle_area(5, 3)
print_square_area(4)
```

Hide pointer operations

- Pointer operations tend to be hard to read and error prone. By isolating them in routines, you can concentrate on the intent of the operation rather than on the mechanics of pointer manipulation.
- Also, if the operations are done in only one place, you can be more certain that the code is correct. If you find a better data type than pointers, you can change the program without traumatizing the code that would have used the pointers.

Code example

```
#include <iostream>
int main() {
    int number = 0; std::cout << "Initial value: " << number << std::endl;
int* ptr = &number
    *ptr = 42;
std::cout << "Updated value: " << number << std::endl    return 0;
}
```

```
#include <iostream>
void setValue(int* ptr, int value) {
    *ptr = value;   }
int main() {
    int number = 0;
std::cout << "Initial value: " << number << std::endl;

setValue(&number, 42);
    std::cout << "Updated value: " << number << std::endl;
return 0;
}
```

Improve portability

- Nonportable capabilities include nonstandard language features, hardware dependencies, operating-system dependencies, and so on.
- To improve the portability of your code, you should isolate any nonportable capabilities into separate functions or modules. This includes:
- Nonstandard language features: Features that may not be available in all Python versions or implementations.
- Hardware dependencies: Code that relies on specific hardware characteristics or configurations.
- Operating system dependencies: Code that depends on specific operating systems or platforms.
- By isolating these nonportable capabilities, you can easily identify and manage future portability work.

Improve performance

- Centralizing code into routines not only simplifies the process of optimizing and profiling but also enhances maintainability and scalability.
- By focusing on a single point of improvement, developers can achieve better performance and easier code management.
- This practice is a fundamental principle in software engineering and contributes to cleaner, more efficient codebases.

HOW THE FUNCTIONS SHOULD BE ?

- The first rule of functions is that they should be small.
- Functions should do one thing. they should do it well they should do it only.
- Classes That Should Just Be Functions or Modules
- Operations That Seem Too Simple to Put Into Routines
- Meaningful name (better to use verbs)

Function should be small

Screen size criteria ?

1980 …



Blocks and Indenting

- This implies that functions should not be large enough to hold nested structures (e.g. if, else ,loops).
- Therefore, the indent level of a function should not be greater than one or two. This, of course, makes the functions easier to read and understand.

Function Name Matters

- Ward's principle: "You know you are working on clean code when each routine turns out to be pretty much what you expected." Half the battle to achieving that principle is choosing good names for small functions that do one thing.
- The smaller and more focused a function is, the easier it is to choose a descriptive name.

GIVE ME EXAMPLES ?

- A long descriptive name is better than a short enigmatic name.
- A long descriptive name is better than a long descriptive comment. Use a naming convention that allows multiple words to be easily read in the function names, and then make use of those multiple words to give the function a name that says what it does.

- Continued…

For queries

Office : B05, RSB

Counselling Hours :

Monday	11:00 am -12:00 pm
Tuesday	12:00 pm -1:00 pm
Thursday	10:15 am -11:15 am

mumer.haroon@imsciences.edu.pk

Muhammad Umer Haroon | LinkedIn